

MLM Tutorial — Generating Twisted Bilayer Structures

This tutorial walks through the **complete workflow** for generating a commensurate twisted bilayer *moiré* supercell with **MLM** — from finding candidate twist angles to writing simulation-ready structure files.

It covers the two core notebooks:

Notebook	Purpose
notebooks/Bi-layer_structure_finder.ipynb	Finds commensurate twist angles
notebooks/Bi-layer-str-writer.ipynb	Builds and writes the atomic structures

Prerequisites

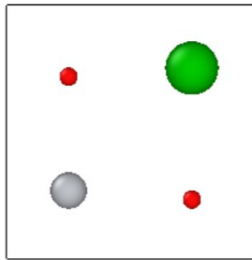
Install MLM and its dependencies from the repository root:

```
pip install -e .
```

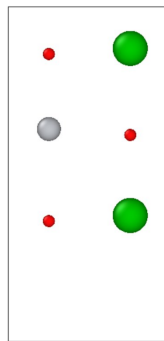
Input requirement: MLM reads your monolayers from **VASP POSCAR** / **.vasp files**.

Prepare one file per layer, each containing a single primitive (or small) unit cell of the

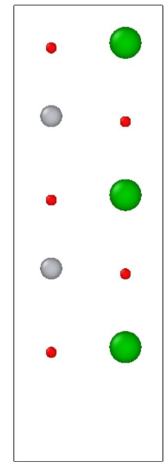
material. These are the only inputs you need to provide. Look at the figure below which shows example of taking a bulk BTO and monolayers of RP1 and RP2 style. This can be achieved using tools like OVITO or text editor.



(a) BTO bulk unit cell



(b) RP1 monolayer



(c) RP2 monolayer

Part 1 — Finding Commensurate Twist Angles

Open [notebooks/Bi-layer_structure_finder.ipynb](#).

Step 1 — Import the modules

```
import MLM.moire_lattice_vector_finder as mlf
import numpy as np
import pandas as pd
from MLM.match import run_and_filter
```

Step 2 — Load your POSCAR files

Point [file_1](#) and [file_2](#) at the monolayer **POSCAR/.vasp** files for your **bottom** and **top** layers.

```
file_1 = "POSCAR_graphene" # bottom layer
file_2 = "POSCAR_graphene" # top layer

lattice_vectors1, atom_type_arr1, dat1 = mlf.read_vasp(file_1)
lattice_vectors2, atom_type_arr2, dat2 = mlf.read_vasp(file_2)
```

`read_vasp` returns the in-plane lattice vectors as a dict with keys 'a', 'b', 'c'.

Homobilayer vs. heterobilayer:

- For a homobilayer (e.g. bilayer graphene), `file_1` and `file_2` are the **same** file.
- For a heterobilayer (e.g. PbTiO_3 on SrTiO_3), simply point the two files at **different**

materials — MLM handles the lattice mismatch automatically.

Step 3 — Run the coincidence search

`run_and_filter` scans a range of twist angles and returns the commensurate matches.

```
result = run_and_filter(
    a1=lattice_vectors1['a'][0:2], # bottom-layer vector 1 (in-plane)
    a2=lattice_vectors1['b'][0:2], # bottom-layer vector 2 (in-plane)
    g1=lattice_vectors2['a'][0:2], # top-layer vector 1 (in-plane)
    g2=lattice_vectors2['b'][0:2], # top-layer vector 2 (in-plane)
    degmin=1.0,
    degmax=45.0,
    degstep=0.01,
    dtol=9e-3,
    nmin=-10,
    nmax=10,
    goodangles=[60.0],
)
```

Key parameters:

Parameter	Meaning
<code>a1, a2</code>	In-plane lattice vectors of the bottom layer
<code>g1, g2</code>	In-plane lattice vectors of the top layer
<code>degmin, degmax</code>	Twist-angle range to scan (degrees)
<code>degstep</code>	Angular resolution of the scan (degrees)
<code>dtol</code>	Coincidence tolerance — how close vectors must match (Å)
<code>nmin, nmax</code>	Integer search range for lattice-vector combinations

goodangles

Target cell angle(s): [60.0]/[120.0] for hexagonal, [90.0] for square

Tip: Start coarse (*degstep=0.1*) to get an overview, then narrow the range and use

degstep=0.01 to pin down precise angles.

Step 4 — Parse the results into a DataFrame

```
df = pd.DataFrame(result[1]) # tabulated matches
df.head()
```

The DataFrame contains the columns:

Column	Description
Theta	Twist angle (degrees)
A1, A2	Supercell lattice vectors
delvec	Cartesian mismatch residual (Å) — your quality metric
delang	Deviation from the target cell angle
norm_A1, norm_A2	Supercell vector lengths (Å)

Step 5 — Filter to the best candidates

Cluster nearby angles and keep the lowest-mismatch entry from each cluster:

```
df_sorted = df.sort_values("delvec")
best = (
    df_sorted
    .groupby(df_sorted["Theta"].round(1)) # cluster angles within ~0.05°
    .first()
    .reset_index(drop=True)
)
best
```

Quality guide: Prefer structures with *delvec < 0.01 Å* for production simulations.

Smaller is better — a smaller delvec means the moiré cell is closer to exactly

commensurate.

Step 6 — Save the results

```
best.to_pickle("graphene_candidates.pkl")
```

You now have a pickle of vetted twist angles and supercell vectors to feed into the structure writer.

Part 2 — Building and Writing the Atomic Structures

Open [notebooks/Bi-layer-str-writer.ipynb](#).

Step 1 — Import the modules

```
import MLM.structure_writer as sw
import numpy as np
import pandas as pd
```

Step 2 — Load the POSCAR files and candidate results

```
file_1 = "POSCAR_graphene"
file_2 = "POSCAR_graphene"

lattice_vectors1, atom_type_arr1, dat1 = sw.read_vasp(file_1)
lattice_vectors2, atom_type_arr2, dat2 = sw.read_vasp(file_2)

candidates = pd.read_pickle("graphene_candidates.pkl")
```

Step 3 — Generate the structure for a chosen candidate

For a selected row (twist angle `theta`, supercell vectors `A1`, `A2`, mismatch `delvec`):

a) Replicate each monolayer to a large enough patch to cover the supercell:

```
new_lattice_vectors, new_total_atoms, replicated_atom, replicated_type_arr =
sw.replicate_atoms(
    a, b, c, atom_data, atom_type_arr, natoms, na, nb, nc
)
```

A safe replication factor is the supercell vector length divided by the unit-cell length, with headroom:

```
factor = int(norm / min(cell_lengths)) * 8
```

b) Rotate the top layer by the twist angle:

```
rotation_matrix = np.array([
    [np.cos(theta_rad), -np.sin(theta_rad)],
    [np.sin(theta_rad), np.cos(theta_rad)],
])
rotated_positions = positions @ rotation_matrix.T
```

c) Set the interlayer z-distance and stack the layers. This is the line that controls how far apart the two layers sit along z:

```
positions2_df['z'] = positions2_df['z'] + interlayer_distance + positions1_df['z'].max()
```

Setting the interlayer distance: *interlayer_distance* (the **3** in the notebook, in Å)

is the gap between the top of the bottom layer and the bottom of the top layer.

Choose it based on your material:

- **van der Waals materials** (graphene, MoS_2): **~3.3–3.5 Å**
- **Perovskite oxides** (SrTiO_3 , PbTiO_3): **~3.8–4.0 Å**

d) Add vacuum and define the out-of-plane cell vector:

```
z_dim = (positions2_df['z'].max() - positions1_df['z'].min()) + 50 # 50 Å vacuum
A3 = [0, 0, z_dim]
```

e) Select the atoms that fall inside the commensurate moiré cell:

```
selected_atoms_df = sw.select_atoms_fractional(concat_vertical, A1, A2, dtol=delvec)
```

Step 4 — Write the output files

Export simulation-ready structures in **VASP** or **LAMMPS** format:

```
sw.write_vasp_ase("POSCAR_TBL_graphene", A1, A2, A3, selected_atoms_df)
sw.write_lammps_ase("data.TBL_graphene", A1, A2, A3, selected_atoms_df)
```

Your twisted bilayer is now ready for DFT (VASP) or MD (LAMMPS) simulation.

Summary of the Full Workflow

1. Prepare monolayer POSCAR / .vasp files (one per layer)



2. Bi-layer_structure_finder.ipynb

- Load POSCARs with `mlf.read_vasp()`
- Scan twist angles with `run_and_filter()`
- Filter candidates by `delvec` and supercell size
- Save results to .pkl



3. Bi-layer-str-writer.ipynb

- Load POSCARs and .pkl candidates
- Replicate, rotate, and stack the layers
- Set the interlayer z-distance
- Select atoms with `select_atoms_fractional()`
- Write VASP or LAMMPS output files

Tips

- **Start with a coarse scan** (e.g. 1° – 30° , `degstep=0.1`) for a quick overview, then narrow the range and use `degstep=0.01` for precise angles.
- **Watch the supercell size.** Large cells (`norm > 50 Å`) contain many thousands of atoms — confirm your DFT/MD code can handle the system before generating.
- **delvec is your quality metric.** Prefer `delvec < 0.01 Å` for production runs.
- **Match good angles to the lattice:** `[60.0]`/`[120.0]` for hexagonal (graphene, MoS_2), `[90.0]` for square (SrTiO_3 , perovskites).
- The replication factor is computed automatically, but you can **override it manually** if the auto value yields too few or too many atoms.

Extending to Three or More Layers

The workflow for **trilayers** (and higher stacks) is essentially the same as the bilayer case above — there is simply *one extra file* threaded through the pipeline. Use the companion notebooks:

Notebook	Purpose
notebooks/trilayer_candidate_finder.ipynb	Finds commensurate twist-angle combinations
notebooks/trilayer_structure_creator.ipynb	Builds and writes the trilayer structures

The one conceptual difference is that a multilayer search is **sequential**: MLM first finds a commensurate supercell for layers 1 and 2 at angle θ_1 , then uses those supercell vectors as the reference lattice for a second coincidence search against layer 3 at angle θ_2 .

In practice this just means you provide a **second twist angle** and a **third POSCAR/.vasp file**, and the structure writer stacks the extra layer with its own interlayer z-distance — exactly as you did for the second layer above. Everything else ([read_vasp](#), filtering by [delvec](#), replicate/rotate/stack, [select_atoms_fractional](#), and VASP/LAMMPS output) works identically.